Task Tracker, Line by Line

React, Node.js and PostgreSQL in Docker and Kubernetes

The node-postgres task tracker explained file by file: the Express API and its storage interface, the React page, the containers and manifests, the tests, and the CI that runs them on every push.

Part	What it covers
0. The big picture	The pieces, the API, the layout
1. The API	Storage interface, validation, routes, Express, PostgreSQL, shutdown
2. The front end	React, a typed API client, nginx, client tests
3. Containers and Kubernetes	Dockerfiles, Compose, StatefulSet, probes, Secrets
4. Tests	Store contract against memory and PostgreSQL, routes over HTTP
5. Keeping it working	SemVer, lock files, npm ci, CI with a database, Dependabot

Every code listing shows the real file and its real line numbers. The notes under a listing refer to those numbers, so you can read the book with the project open beside it.

Contents

The big picture 4

0.1 The pieces 5

0.2 The API 5

0.3 The layout 5

The API 7

1.1 store.ts: what storage must do 8

1.2 validation.ts: what valid input is 8

1.3 routes.ts: HTTP in, store calls out 9

1.4 app.ts: assembling Express 11

1.5 db.ts: PostgreSQL 11

1.6 index.ts: starting and stopping 13

1.7 Interview questions 13

The front end 15

2.1 Where the page comes from 16

2.2 The API client 16

2.3 The component 17

2.4 Development and production 19

2.5 Testing the client without a browser 20

2.6 Interview questions 21

Containers and Kubernetes 22

3.1 The API image 23

3.2 Docker Compose 23

3.3 Kubernetes 24

3.4 Interview questions 27

Tests 28

4.1 The store contract 29

4.2 The in-memory store 30

4.3 Routes over HTTP 31

4.4 Running them 32

4.5 Interview questions 32

Keeping it working 33

5.1 package.json 34

5.2 Lock files and npm ci 34

5.3 The workflow 35

5.4 Dependabot 37

5.5 Interview questions 38

Part 0

The big picture

A task tracker in three containers: a React page served by nginx, an Express API in TypeScript, and PostgreSQL. It runs the same way under Docker Compose and Kubernetes, and every push is tested in CI.

0.1 The pieces

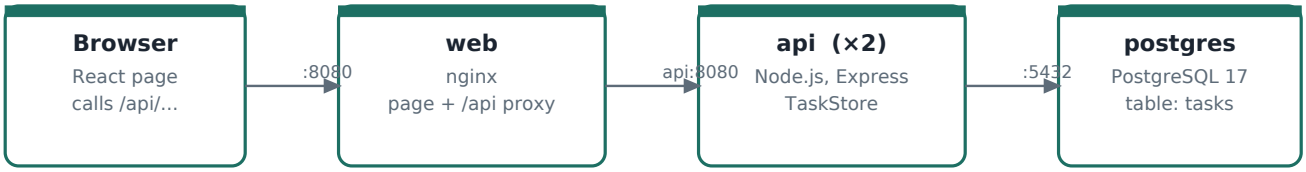


Figure 0.1. One request: the browser asks nginx, nginx forwards /api to the API, the API queries PostgreSQL.

Piece	Built with	Job
web	React 19, Vite, nginx	Serves the page; forwards `/api/` to the API
api	Node.js 22, Express 5, pg, TypeScript	REST endpoints, validation, SQL
postgres	PostgreSQL 17	Stores tasks; its data survives restarts in a volume

0.2 The API

Method and path	Does	Answers
GET /api/tasks	List tasks, oldest first	200 + array
POST /api/tasks	Create from `{ title }`	201 + task; 400 if the title is bad
PATCH /api/tasks/:id	Change `title` and/or `done`	200 + task; 400; 404
DELETE /api/tasks/:id	Delete	204; 400; 404
GET /api/health/live	Is the process up?	200
GET /api/health/ready	Can it reach the database?	200 or 503
GET /api/info	Which pod answered	200 + `{ hostname }`

0.3 The layout

Path	What's there
`api/src/`	store.ts (the storage interface), validation.ts, routes.ts, app.ts, db.ts (PostgreSQL), index.ts
`api/tests/`	Validation, routes over HTTP, and one store contract run against memory and PostgreSQL
`frontend/`	React app, its tests, nginx.conf and Dockerfile
`k8s/`	Kubernetes manifests, applied with `kubectl apply -k k8s`
`docker-compose.yml`	All three containers locally
`walkthrough/`	This book and the script that builds it

One idea shapes the API: **the routes don't know where tasks are stored**. They talk to a `TaskStore` interface. The running app passes PostgreSQL; the tests pass an in-memory store. A shared contract test checks that both behave the same.

> This book is generated from the source by `walkthrough/build_walkthrough.py`. Every listing is the real file with its real line numbers, and the bold "Line n" notes refer to them.

Part 1

The API

Six small files, each with one job. Read them in this order: what storage must do, what valid input is, the routes, the app, PostgreSQL, and startup.

1.1 store.ts: what storage must do

TypeScript api/src/store.ts, lines 1–25

```ts
/** A task as the API returns it. */
export type TaskRow = {
  id: number;
  title: string;
  done: boolean;
  createdAt: Date;
};

export type TaskChanges = { title?: string; done?: boolean };

/**
 * Everything the routes need from storage. PostgreSQL implements it in db.ts; the tests also ru
    n
 * the routes and the same contract against an in-memory version, so the HTTP layer can be teste
    d
 * without a database.
 */
export interface TaskStore {
  list(): Promise<TaskRow[]>;
  create(title: string): Promise<TaskRow>;
  /** Undefined when no task has that id. */
  update(id: number, changes: TaskChanges): Promise<TaskRow | undefined>;
  /** False when no task has that id. */
  remove(id: number): Promise<boolean>;
  /** Throws if the database can't be reached (used by the readiness probe). */
  ping(): Promise<void>;
}
```

Line 9 A PATCH body: both fields optional. Missing means "leave as is", which is different from `false`.

Line 20 "Not found" is part of the type, not an exception. The route turns it into a 404.

Line 24 For the readiness probe: throws if the database is unreachable.

This is the seam that makes the API testable. Express and HTTP are on one side, SQL on the other, and neither needs the other to be tested.

1.2 validation.ts: what valid input is

TypeScript api/src/validation.ts, lines 1–14

```ts
/** Input rules for the task API, kept free of Express so they can be unit-tested directly. */

export const MAX_TITLE = 200;

/** A route id: a positive whole number, or undefined. */
export function parseId(value: unknown): number | undefined {
  const id = Number(value);
  return Number.isInteger(id) && id >= 1 ? id : undefined;
}

/** A title must be a string with something other than spaces in it, at most 200 characters. */
export function validTitle(value: unknown): value is string {
  return typeof value === "string" && value.trim().length > 0 && value.length <= MAX_TITLE;
}
```

Line 7 Route parameters are always strings. `Number("42")` is 42; `Number("abc")` is `NaN`; `Number("")` is 0.

Line 8 `isInteger` rejects `NaN` and 1.5; `>= 1` rejects 0 and negatives. SERIAL ids start at 1.

Line 12 A **type guard**: when it returns true, TypeScript treats `value` as a string afterwards. The parameter is `unknown` because a JSON body can hold anything.

Line 13 Not blank, and at most 200 characters, matching the `VARCHAR(200)` column. The length is measured before trimming, so " x" padded past 200 is refused even though the trimmed text would fit: strict, and simple.

No Express here, so these rules are tested with plain function calls.

1.3 routes.ts: HTTP in, store calls out

TypeScript api/src/routes.ts, lines 6–25

```
export function createRouter(tasks: TaskStore): Router {
  const router = Router();

  // Liveness: the process is up. Kubernetes restarts the pod if this fails.
  router.get("/health/live", (_req, res) => {
    res.json({ status: "ok" });
  });

  // Readiness: the database is reachable. Kubernetes stops sending traffic if this fails.
  router.get("/health/ready", async (_req, res) => {
    try {
      await tasks.ping();
      res.json({ status: "ok" });
    } catch {
      res.status(503).json({ status: "database unavailable" });
    }
  });

  // Which pod answered. Inside Kubernetes the hostname is the pod name.
  router.get("/info", (_req, res) => {
```

Line 10 **Liveness** never touches the database. If it did, a database outage would make Kubernetes restart every API pod, which fixes nothing.

Line 15 **Readiness** does. While it fails, Kubernetes stops sending this pod traffic but leaves it running.

TypeScript api/src/routes.ts, lines 29–31

```
  router.get("/tasks", async (_req, res) => {
    res.json(await tasks.list());
  });
```

TypeScript api/src/routes.ts, lines 33–40

```
  router.post("/tasks", async (req, res) => {
    const { title } = req.body ?? {};
    if (!validTitle(title)) {
      res.status(400).json({ error: "title is required (1-200 characters)" });
      return;
    }
    res.status(201).json(await tasks.create(title.trim()));
```

```
40  });
```

Line 34 `req.body` is undefined without a JSON body; `?? {}` makes destructuring safe.

Line 39 201 Created with the new task. `title` is known to be a string here, thanks to the type guard.

TypeScript api/src/routes.ts, lines 42–63

```
42  router.patch("/tasks/:id", async (req, res) => {
43    const id = parseId(req.params.id);
44    if (id === undefined) {
45      res.status(400).json({ error: "id must be a positive integer" });
46      return;
47    }
48    const { title, done } = req.body ?? {};
49    if (title !== undefined && !validTitle(title)) {
50      res.status(400).json({ error: "title must be 1-200 characters" });
51      return;
52    }
53    if (done !== undefined && typeof done !== "boolean") {
54      res.status(400).json({ error: "done must be true or false" });
55      return;
56    }
57    const updated = await tasks.update(id, { title: title?.trim(), done });
58    if (!updated) {
59      res.status(404).json({ error: "task not found" });
60      return;
61    }
62    res.json(updated);
63  });
```

Line 49 Each field is checked only if it was sent.

Line 53 A string `"true"` is refused: the API takes JSON booleans, not text that looks like one.

Line 57 `title?.trim()` keeps an absent title absent.

TypeScript api/src/routes.ts, lines 65–76

```
65  router.delete("/tasks/:id", async (req, res) => {
66    const id = parseId(req.params.id);
67    if (id === undefined) {
68      res.status(400).json({ error: "id must be a positive integer" });
69      return;
70    }
71    if (!(await tasks.remove(id))) {
72      res.status(404).json({ error: "task not found" });
73      return;
74    }
75    res.status(204).end();
76  });
```

Validation errors are always 400 with `{ error }`, a missing task is always 404, and a delete returns 204 with no body. The front end relies on all three.

1.4 app.ts: assembling Express

TypeScript api/src/app.ts, lines 1–18

```ts
import express, { type Express, type NextFunction, type Request, type Response } from "express";
import { createRouter } from "./routes.js";
import type { TaskStore } from "./store.js";

/** The Express app, given where tasks are stored. index.ts passes PostgreSQL; tests pass memory
      . */
export function createApp(tasks: TaskStore): Express {
  const app = express();
  app.use(express.json());
  app.use("/api", createRouter(tasks));

  // Express 5 forwards errors from async handlers here.
  app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
    console.error(err);
    res.status(500).json({ error: "internal server error" });
  });

  return app;
}
```

Line 6 A function, not a module-level `app`: each caller chooses the store. That's **dependency injection** without a framework.

Line 8 Parses JSON request bodies into `req.body`.

Line 12 Four parameters mark an error handler. Express 5 sends rejected promises from async handlers here (Express 4 needed a wrapper). The client gets a generic message; the details go to the log, not to the caller.

1.5 db.ts: PostgreSQL

TypeScript api/src/db.ts, lines 4–8

```ts
// pg reads PGHOST, PGPORT, PGUSER, PGPASSWORD and PGDATABASE from the environment,
// which docker-compose.yml and the Kubernetes manifests set.
export function createPool(): pg.Pool {
  return new pg.Pool({ max: 10 });
}
```

A **connection pool** keeps up to ten connections open and lends them to queries, rather than connecting for every request. With no settings passed, `pg` reads `PGHOST`, `PGUSER` and the rest from the environment, which Compose, Kubernetes and CI set.

TypeScript api/src/db.ts, line 10

```ts
const COLUMNS = `id, title, done, created_at AS "createdAt"`;
```

The column is `created_at` in SQL (snake_case, the PostgreSQL convention) and `createdAt` in JSON. The alias does the conversion in one place, and the quotes keep the capital A: PostgreSQL lower-cases unquoted names.

TypeScript api/src/db.ts, lines 12–30

```
12  /** Creates the table if needed. Retries because the database may still be starting up. */
13  export async function migrate(pool: pg.Pool, attempts = 30): Promise<void> {
14    for (let i = 1; ; i++) {
15      try {
16        await pool.query(`
17          CREATE TABLE IF NOT EXISTS tasks (
18            id         SERIAL PRIMARY KEY,
19            title      VARCHAR(200) NOT NULL,
20            done       BOOLEAN NOT NULL DEFAULT FALSE,
21            created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
22          )`);
23        return;
24      } catch (err) {
25        if (i >= attempts) throw err;
26        console.log(`Database not ready (attempt ${i}/${attempts}), retrying in 2s...`);
27        await new Promise((r) => setTimeout(r, 2000));
28      }
29    }
30  }
```

Line 17 Safe to run on every start: the second time, nothing happens.

Line 18 `SERIAL` is an integer filled from a sequence. Sequences never go backwards, so a deleted id isn't reused; a contract test checks that.

Line 21 `TIMESTAMPTZ` stores an instant, not a wall-clock time, so the time zone of the server doesn't matter.

Line 25 The database may still be starting. Retry every 2 seconds, up to a minute, then give up and let the process exit.

TypeScript api/src/db.ts, lines 32–65

```
32  /** The task store backed by PostgreSQL. */
33  export function pgStore(pool: pg.Pool): TaskStore {
34    return {
35      async list() {
36        const { rows } = await pool.query<TaskRow>(`SELECT ${COLUMNS} FROM tasks ORDER BY id`);
37        return rows;
38      },
39      async create(title) {
40        const { rows } = await pool.query<TaskRow>(
41          `INSERT INTO tasks (title) VALUES ($1) RETURNING ${COLUMNS}`,
42          [title],
43        );
44        return rows[0];
45      },
46      async update(id: number, changes: TaskChanges) {
47        const { rows } = await pool.query<TaskRow>(
48          `UPDATE tasks
49              SET title = COALESCE($2, title),
50                  done  = COALESCE($3, done)
51            WHERE id = $1
52        RETURNING ${COLUMNS}`,
53          [id, changes.title ?? null, changes.done ?? null],
54        );
55        return rows[0];
56      },
57      async remove(id) {
58        const { rowCount } = await pool.query(`DELETE FROM tasks WHERE id = $1`, [id]);
59        return (rowCount ?? 0) > 0;
60      },
61      async ping() {
62        await pool.query("SELECT 1");
```

```
    },
  };
}
```

Line 41 `$1` is a **parameter**: the title is sent separately from the SQL text, so a title like `'); DROP TABLE tasks; --` is just a title. `RETURNING` gives back the new row in the same round trip. (`${COLUMNS}` is inserted by JavaScript, but it's a constant, never user input.)

Line 49 `COALESCE` takes the first non-null: a missing field is sent as `null` and the column keeps its value. One statement handles every combination of fields.

Line 44 No row means no task with that id, so this is `undefined`, as the interface promises.

Line 59 How many rows the DELETE removed: 1 or 0.

1.6 index.ts: starting and stopping

TypeScript api/src/index.ts, lines 1–16

```
import { createApp } from "./app.js";
import { createPool, migrate, pgStore } from "./db.js";

const port = Number(process.env.PORT ?? 8080);

const pool = createPool();
await migrate(pool);
const server = createApp(pgStore(pool)).listen(port, () => console.log(`API listening on :${port
      }`));

// Kubernetes sends SIGTERM before stopping a pod: finish in-flight requests, then exit.
for (const signal of ["SIGTERM", "SIGINT"] as const) {
  process.on(signal, () => {
    console.log(`${signal} received, shutting down`);
    server.close(() => pool.end().then(() => process.exit(0)));
  });
}
```

Line 7 **Top-level await**, allowed in ES modules (`"type": "module"` in package.json). The server doesn't listen until the table exists.

Line 14 A **graceful shutdown**: stop accepting connections, let requests in progress finish, close the pool, exit. Kubernetes sends SIGTERM and waits up to 30 seconds before killing the pod.

1.7 Interview questions

Why put storage behind an interface in such a small app?

So the HTTP layer can be tested without a database, and so the same tests can run against the real one. It costs one small file.

How does this API prevent SQL injection?

Every value goes through a parameter (`$1`, `$2`). The driver sends values separately from the SQL text, so they're never parsed as SQL.

What's the difference between liveness and readiness?

Liveness: should this process be restarted? Readiness: should it get traffic right now? A database outage should fail readiness only.

Why validate in the route when the column is already VARCHAR(200)?

To answer with a clear 400 instead of a database error turning into a 500, and to refuse bad input before using a connection.

Part 2

The front end

One React component, a typed API client, and nginx in front. The page never knows which backend it's talking to: every call goes to /api, and whoever answers there is the API. That's why this front end is shared, file for file, by the Node and the .NET versions of the tracker.

2.1 Where the page comes from

HTML frontend/index.html, lines 1–12

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Task Tracker</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

TSX frontend/src/main.tsx, lines 1–10

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Line 6 React takes over the empty `<div id="root">`. The `!` tells TypeScript "this isn't null"; index.html guarantees it.

Line 7 In development only, React runs effects twice to flush out code that isn't safe to re-run. It's why the initial load in App.tsx must be harmless to repeat (it is: it just fetches).

TypeScript frontend/src/config.ts, lines 1–2

```ts
// Shown in the header so it's obvious which backend you're talking to.
export const STACK_LABEL = "React · Node.js · PostgreSQL";
```

The only file that differs between the two projects. This one says `React · Node.js · PostgreSQL`.

2.2 The API client

TypeScript frontend/src/api.ts, lines 1–6

```ts
export interface Task {
  id: number;
  title: string;
  done: boolean;
  createdAt: string;
}
```

`createdAt` is a `string` here but a date on the server: JSON has no date type, so it arrives as ISO text. The type says what's really on the wire.

TypeScript frontend/src/api.ts, lines 8–18

```ts
async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`/api${path}`, {
    headers: { "Content-Type": "application/json" },
    ...init,
  });
  if (!res.ok) {
    const body = await res.json().catch(() => null);
    throw new Error(body?.error ?? `Request failed (${res.status})`);
  }
  return res.status === 204 ? (undefined as T) : res.json();
}
```

Line 9 A relative URL: the same origin that served the page. In development Vite forwards `/api`; in Docker and Kubernetes, nginx does.

Line 11 Spread after the default headers, so a caller could override them; method and body come from here.

Line 13 `fetch` only rejects on network failure. A 400 or 500 is a successful fetch, so the status has to be checked.

Line 14 Use the API's own `{ error }` message if there is one. A proxy's 502 page isn't JSON, so parsing can fail; then fall back to the status.

Line 17 204 No Content has no body, and calling `.json()` on it would throw.

TypeScript frontend/src/api.ts, lines 20–28

```ts
export const api = {
  list: () => request<Task[]>("/tasks"),
  create: (title: string) =>
    request<Task>("/tasks", { method: "POST", body: JSON.stringify({ title }) }),
  update: (id: number, changes: Partial<Pick<Task, "title" | "done">>) =>
    request<Task>(`/tasks/${id}`, { method: "PATCH", body: JSON.stringify(changes) }),
  remove: (id: number) => request<void>(`/tasks/${id}`, { method: "DELETE" }),
  info: () => request<{ hostname: string }>("/info"),
};
```

`Partial<Pick<Task, "title" | "done">>` builds the update type from `Task`: pick two fields, make both optional. Rename `done` in `Task` and this follows; you can't send a field the API doesn't take.

2.3 The component

TSX frontend/src/App.tsx, lines 5–10

```tsx
export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [title, setTitle] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [servedBy, setServedBy] = useState<string | null>(null);
```

Five pieces of state. Each `useState` returns the current value and a setter; calling the setter re-renders the component with the new value.

TSX frontend/src/App.tsx, lines 12–29

```tsx
async function run(action: () => Promise<void>) {
  setError(null);
  try {
    await action();
  } catch (e) {
    setError(e instanceof Error ? e.message : String(e));
  }
}

async function refresh() {
  setTasks(await api.list());
  // Which API pod answered: handy for seeing Kubernetes load-balance across replicas.
  setServedBy((await api.info()).hostname);
}

useEffect(() => {
  run(refresh).finally(() => setLoading(false));
}, []);
```

Line 12 Every user action goes through `run`: clear the old error, try, and show any failure. Error handling is written once.

Line 24 With two API replicas behind a Kubernetes Service, refreshing shows the pod name change: load balancing you can see.

Line 29 An empty dependency list: run once, when the component first appears. `finally` clears the loading state whether the load worked or not.

TSX frontend/src/App.tsx, lines 31–40

```tsx
function add(e: FormEvent) {
  e.preventDefault();
  const trimmed = title.trim();
  if (!trimmed) return;
  run(async () => {
    await api.create(trimmed);
    setTitle("");
    await refresh();
  });
}
```

`e.preventDefault()` stops the browser's own form submit, which would reload the page. After a change, the page re-reads the list from the API rather than editing its own copy: one extra request, but what you see is what the database has.

TSX frontend/src/App.tsx, lines 64–90

```tsx
      {loading ? (
        <p className="muted">Loading…</p>
      ) : tasks.length === 0 ? (
        <p className="muted">No tasks yet. Add one above.</p>
      ) : (
        <ul>
          {tasks.map((t) => (
            <li key={t.id} className={t.done ? "done" : ""}>
              <label>
                <input
                  type="checkbox"
                  checked={t.done}
                  onChange={() => run(async () => { await api.update(t.id, { done: !t.done }); a
wait refresh(); })}
```

```
                />
                <span>{t.title}</span>
              </label>
              <button
                className="delete"
                aria-label={`Delete ${t.title}`}
                onClick={() => run(async () => { await api.remove(t.id); await refresh(); })}
              >
                ×
              </button>
            </li>
          ))}
        </ul>
      )}
```

Line 71 key lets React match list items between renders. The database id is stable; an array index would shift when a task is deleted.

Line 82 The button shows only ×; the label tells a screen reader which task it deletes.

2.4 Development and production

TypeScript frontend/vite.config.ts, lines 1–11

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// In local dev (npm run dev), /api calls are forwarded to an API running on :8080.
// In Docker and Kubernetes, nginx does this forwarding instead (see nginx.conf).
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: { "/api": "http://localhost:8080" },
  },
});
```

nginx frontend/nginx.conf, lines 1–17

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  # Forward API calls to the "api" service. Docker Compose and Kubernetes
  # both resolve that name to the API container(s) via their internal DNS.
  location /api/ {
    proxy_pass http://api:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }

  location / {
    try_files $uri /index.html;
  }
}
```

Line 9 "api" is a DNS name: the compose service in Docker, the Service in Kubernetes. Neither the page nor nginx needs an IP address.

Line 15 A file if it exists, otherwise the app. Standard for single-page apps, so a deep link doesn't 404.

Dockerfile frontend/Dockerfile, lines 1–14

```dockerfile
# ---- build stage: compile the React app to static files ----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
# npm ci installs exactly what package-lock.json records.
RUN npm ci
COPY . .
RUN npm run build

# ---- runtime stage: tiny nginx image serving the static files ----
FROM nginx:1.31-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

A **multi-stage build**. The first stage has Node, every dev dependency and the source; it produces a folder of static files. The final image is nginx plus that folder: no Node, no `node_modules`, a few tens of megabytes. `npm ci` installs exactly what `package-lock.json` records.

Under Docker Compose the page is on `http://localhost:8080`.

2.5 Testing the client without a browser

TypeScript frontend/tests/api.test.ts, lines 10–17

```typescript
/** Makes every fetch return this status and body, and remember what it was asked. */
function respond(status: number, body?: unknown) {
  calls = [];
  globalThis.fetch = (async (url: string, init?: RequestInit) => {
    calls.push({ url, init });
    return new Response(body === undefined ? null : JSON.stringify(body), { status });
  }) as typeof fetch;
}
```

The test replaces the global `fetch` with a fake that records the request and returns a canned `Response` (Node 18+ has the same `Response` class as the browser). `afterEach` puts the real one back.

TypeScript frontend/tests/api.test.ts, lines 54–55

```typescript
test("an error response throws the API's own message", async () => {
  respond(400, { error: "title is required (1-200 characters)" });
```

These check the edges of the client: the URL and method of each call, the JSON body, 204 handling, and both error paths. Run them with `npm test` in `frontend/`.

2.6 Interview questions

Why does fetch not throw on a 404?

It only rejects when there's no response at all (network error, CORS). Any HTTP response, including errors, resolves; you check `res.ok` or the status yourself.

Why proxy /api instead of calling the API's URL from the browser?

Same origin: no CORS, no API address baked into the build, and the same image works in Compose, Kubernetes or anywhere the name "api" resolves.

Why not update the task list locally after a change?

It's simpler and always matches the database. For a busier app you'd update optimistically and roll back on error.

Part 3

Containers and Kubernetes

The same three images run under Docker Compose on a laptop and under Kubernetes. What changes between the two is only how they find each other and where the settings come from.

3.1 The API image

Dockerfile api/Dockerfile, lines 1–20

```dockerfile
# ---- build stage: compile TypeScript ----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
# npm ci installs exactly what package-lock.json records (and fails if it disagrees with package
      .json).
RUN npm ci
COPY . .
RUN npm run build

# ---- runtime stage: production dependencies + compiled JS only ----
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
# Don't run as root.
USER node
EXPOSE 8080
CMD ["node", "dist/index.js"]
```

Line 4 Copy only the dependency files first. Docker caches each step, so while they don't change, `npm ci` isn't re-run when you edit the code.

Line 15 The runtime image gets production dependencies only: no TypeScript, no tsx, no type packages.

Line 18 The official image has a `node` user. A process that's broken into can't act as root.

3.2 Docker Compose

YAML docker-compose.yml, lines 1–41

```yaml
# Runs the whole app locally with Docker: http://localhost:8080
#   docker compose up --build
name: tasks-node

services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: tasks
      POSTGRES_PASSWORD: tasks-dev-password
      POSTGRES_DB: tasks
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U tasks -d tasks"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: ./api
    image: tasks-node-api:1.0
    environment:
      PGHOST: db
      PGUSER: tasks
      PGPASSWORD: tasks-dev-password
```

```
27        PGDATABASE: tasks
28      depends_on:
29        db:
30          condition: service_healthy
31
32    web:
33      build: ./frontend
34      image: tasks-node-web:1.0
35      ports:
36        - "8080:80"
37      depends_on:
38        - api
39
40  volumes:
41    db-data:
```

Line 13 A named volume: the data outlives the container. `docker compose down -v` deletes it.

Line 15 Compose runs this to decide when the database is healthy...

Line 30 ...and starts the API only after that. The API retries anyway, because Kubernetes has no such ordering.

Line 36 Only the web container is published. The API and database are reachable only inside Compose's network.

3.3 Kubernetes

YAML k8s/kustomization.yaml, lines 1–10

```
1  # Apply everything with:  kubectl apply -k k8s
2  apiVersion: kustomize.config.k8s.io/v1beta1
3  kind: Kustomization
4  namespace: tasks-node
5  resources:
6    - namespace.yaml
7    - db-secret.yaml
8    - postgres.yaml
9    - api.yaml
10   - web.yaml
```

`kubectl apply -k k8s` applies all five files and puts everything in the `tasks-node` namespace, so `kubectl delete namespace tasks-node` removes it all.

YAML k8s/postgres.yaml, lines 1–58

```
1  # Headless service gives the database a stable DNS name: "postgres".
2  apiVersion: v1
3  kind: Service
4  metadata:
5    name: postgres
6  spec:
7    clusterIP: None
8    selector:
9      app: postgres
10   ports:
11     - port: 5432
12 ---
13 # StatefulSet + volumeClaimTemplate: the data survives pod restarts.
14 apiVersion: apps/v1
15 kind: StatefulSet
16 metadata:
17   name: postgres
18 spec:
```

```
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:17-alpine
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: db-credentials
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
          readinessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

Line 7 A **headless Service**: just a DNS name, `postgres`, pointing at the pod.

Line 15 Unlike a Deployment, a StatefulSet gives its pod a stable identity and its own volume.

Line 39 A subfolder of the volume: PostgreSQL refuses to start in a folder that isn't empty, and some volumes come with a `lost+found` folder.

Line 51 Kubernetes creates a 1 GiB volume for the pod and re-attaches it after a restart.

YAML k8s/api.yaml, lines 1–56

```
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector:
    app: api
  ports:
    - port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
```

```
17    replicas: 2
18    selector:
19      matchLabels:
20        app: api
21    template:
22      metadata:
23        labels:
24          app: api
25      spec:
26        containers:
27          - name: api
28            image: tasks-node-api:1.0
29            # Uses the image you built locally; Docker Desktop's cluster shares Docker's images.
30            imagePullPolicy: IfNotPresent
31            ports:
32              - containerPort: 8080
33            env:
34              - name: PGHOST
35                value: postgres
36              - name: PGUSER
37                valueFrom: { secretKeyRef: { name: db-credentials, key: POSTGRES_USER } }
38              - name: PGPASSWORD
39                valueFrom: { secretKeyRef: { name: db-credentials, key: POSTGRES_PASSWORD } }
40              - name: PGDATABASE
41                valueFrom: { secretKeyRef: { name: db-credentials, key: POSTGRES_DB } }
42            # The API waits for the database before it starts listening. The startup probe
43            # gives it up to 150s to do that before liveness checks take over.
44            startupProbe:
45              httpGet: { path: /api/health/live, port: 8080 }
46              periodSeconds: 5
47              failureThreshold: 30
48            livenessProbe:
49              httpGet: { path: /api/health/live, port: 8080 }
50              periodSeconds: 10
51            readinessProbe:
52              httpGet: { path: /api/health/ready, port: 8080 }
53              periodSeconds: 5
54            resources:
55              requests: { cpu: 50m, memory: 64Mi }
56              limits: { memory: 256Mi }
```

Line 17 Two API pods. The Service spreads requests across them; the page's footer shows which one answered.

Line 30 Use the image built locally rather than looking for it in a registry.

Line 37 Credentials come from a Secret rather than being written into the Deployment.

Line 44 Up to 150 seconds (30 × 5) to start, while the API waits for the database. Liveness checks begin only after this succeeds, so a slow start isn't mistaken for a hang.

Line 56 A pod over its memory limit is killed and restarted. The request is what the scheduler reserves.

YAML k8s/web.yaml, lines 1–12

```
1  # LoadBalancer: Docker Desktop publishes this on http://localhost:8080
2  apiVersion: v1
3  kind: Service
4  metadata:
5    name: web
6  spec:
7    type: LoadBalancer
8    selector:
9      app: web
10   ports:
11     - port: 8080
```

```
12        targetPort: 80
```

Docker Desktop publishes a `LoadBalancer` Service on `localhost`. In a cloud, the same manifest creates a real load balancer.

YAML k8s/db-secret.yaml, lines 1–11

```yaml
# Demo credentials only. In a real cluster these would come from a secret
# manager (or be created with `kubectl create secret`), not live in git.
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  POSTGRES_USER: tasks
  POSTGRES_PASSWORD: tasks-k8s-password
  POSTGRES_DB: tasks
```

3.4 Interview questions

Why a StatefulSet for the database and a Deployment for the API?

The API's pods are interchangeable and keep nothing; the database pod needs the same volume every time it restarts.

Why multi-stage Docker builds?

The build tools stay in the first stage. The final image is smaller, starts faster, and has less in it to attack.

The API starts before the database is ready. What happens?

migrate() retries for a minute; the startup probe gives it 150 seconds; readiness keeps traffic away until the database answers.

Part 4

Tests

Three layers: plain functions, the store contract (against memory and PostgreSQL), and every route over real HTTP. 30 tests in the API (the 8-test contract runs twice), 6 in the front end.

File	Tests	Needs
`tests/validation.test.ts`	`parseId` and `validTitle`	nothing
`tests/memory.test.ts`	The store contract, in memory	nothing
`tests/postgres.test.ts`	The store contract, in PostgreSQL	a database named *_test (skipped otherwise)
`tests/routes.test.ts`	Every route over HTTP	nothing (in-memory store)

They use Node's built-in runner, `node:test`, with `tsx` so TypeScript runs without a build step: `npm test`. `npm run typecheck` type-checks the app and the tests together using `tsconfig.test.json`.

4.1 The store contract

TypeScript api/tests/contract.ts, lines 5–15

```
/**
 * The behaviour every TaskStore must have. memory.test.ts runs it against the in-memory store,
 * postgres.test.ts against a real database, so the two can't quietly drift apart.
 *
 * @param fresh returns an empty store; called before every test.
 */
export function storeContract(name: string, fresh: () => Promise<TaskStore>, options: { skip?: s
      tring | false } = {}) {
  const t = (title: string, fn: (store: TaskStore) => Promise<void>) =>
    test(`${name}: ${title}`, { skip: options.skip }, async () => fn(await fresh()));

  t("starts empty", async (store) => {
```

`storeContract` doesn't run tests itself; it *defines* them for whatever store `fresh` makes. Each test gets a new, empty store, so the order they run in doesn't matter.

TypeScript api/tests/contract.ts, lines 35–43

```
  t("update changes only the fields given", async (store) => {
    const task = await store.create("Write tests");
    const done = await store.update(task.id, { done: true });
    assert.equal(done?.title, "Write tests");
    assert.equal(done?.done, true);
    const renamed = await store.update(task.id, { title: "Write more tests" });
    assert.equal(renamed?.title, "Write more tests");
    assert.equal(renamed?.done, true, "done stays as it was");
  });
```

TypeScript api/tests/contract.ts, lines 56–61

```
  t("ids are not reused after a delete", async (store) => {
    const a = await store.create("a");
    await store.remove(a.id);
    const b = await store.create("b");
    assert.notEqual(b.id, a.id);
  });
```

Rules like these are easy to get subtly different in a fake. Running the same tests against both stores is what lets the route tests trust the in-memory one.

TypeScript api/tests/memory.test.ts, lines 1–4

```ts
import { storeContract } from "./contract.js";
import { memoryStore } from "./memoryStore.js";

storeContract("memory", async () => memoryStore());
```

TypeScript api/tests/postgres.test.ts, lines 1–29

```ts
import { after } from "node:test";
import { createPool, migrate, pgStore } from "../src/db.js";
import { storeContract } from "./contract.js";

// These tests empty the tasks table before each one, so they only run against a database whose
// name ends in "_test" (CI starts one; see the README for running it locally). Anything else is
// skipped, which keeps `npm test` from ever wiping your real tasks.
const database = process.env.PGDATABASE ?? "";
const skip = database.endsWith("_test") ? false : `set PGDATABASE to a database ending in _test
      (now "${database}")`;

const pool = skip ? undefined : createPool();
let migrated = false;

storeContract(
  "postgres",
  async () => {
    if (!migrated) {
      await migrate(pool!, 5);
      migrated = true;
    }
    await pool!.query("TRUNCATE tasks RESTART IDENTITY");
    return pgStore(pool!);
  },
  { skip },
);

after(async () => {
  await pool?.end();
});
```

Line 9 A safety rule: these tests empty the table, so they refuse to run against any database whose name doesn't end in `_test`. Running `npm test` with your Compose settings can't wipe your tasks.

Line 21 Empty table, ids from 1 again, before every test.

Line 28 Close the pool, or Node waits on the open connections and the test run never exits.

4.2 The in-memory store

TypeScript api/tests/memoryStore.ts, lines 3–37

```ts
/**
 * An in-memory TaskStore for tests. It follows the same contract as PostgreSQL (tests/contract.
     ts
 * checks both), so route tests that use it are testing the real behaviour, minus the network.
 */
export function memoryStore(options: { down?: boolean } = {}): TaskStore {
  let rows: TaskRow[] = [];
  let nextId = 1;
  const copy = (r: TaskRow): TaskRow => ({ ...r, createdAt: new Date(r.createdAt) });

  return {
    async list() {
```

```
14        return rows.map(copy);
15      },
16      async create(title) {
17        const row = { id: nextId++, title, done: false, createdAt: new Date() };
18        rows.push(row);
19        return copy(row);
20      },
21      async update(id, changes) {
22        const row = rows.find((r) => r.id === id);
23        if (!row) return undefined;
24        if (changes.title !== undefined) row.title = changes.title;
25        if (changes.done !== undefined) row.done = changes.done;
26        return copy(row);
27      },
28      async remove(id) {
29        const before = rows.length;
30        rows = rows.filter((r) => r.id !== id);
31        return rows.length < before;
32      },
33      async ping() {
34        if (options.down) throw new Error("database unavailable");
35      },
36    };
37  }
```

Line 10 Hand out copies. PostgreSQL returns new objects every time; if this returned its own rows, a caller could change stored data by editing a result.

Line 34 A switch to simulate an outage, for the readiness test.

4.3 Routes over HTTP

TypeScript api/tests/routes.test.ts, lines 14–26

```
14  before(async () => {
15    // The app reads `store` through this wrapper, so each test can swap in a fresh one.
16    const current: TaskStore = {
17      list: () => store.list(),
18      create: (t) => store.create(t),
19      update: (id, c) => store.update(id, c),
20      remove: (id) => store.remove(id),
21      ping: () => store.ping(),
22    };
23    server = createApp(current).listen(0);
24    await new Promise<void>((resolve) => server.once("listening", resolve));
25    base = `http://127.0.0.1:${(server.address() as AddressInfo).port}/api`;
26  });
```

Line 16 The app is created once, but each test wants a fresh store. This wrapper forwards to whatever `store` is now, so `beforeEach` can swap it.

Line 23 Port 0: the operating system picks a free port. Tests never clash with a running app or with each other.

TypeScript api/tests/routes.test.ts, lines 56–63

```
56  test("POST /tasks rejects a missing, blank or too-long title with 400", async () => {
57    for (const body of [{}, { title: "   " }, { title: "x".repeat(201) }, { title: 5 }]) {
58      const res = await call("POST", "/tasks", body);
59      assert.equal(res.status, 400, JSON.stringify(body));
60      assert.match(res.body.error, /title/);
61    }
62    assert.deepEqual((await call("GET", "/tasks")).body, [], "nothing was stored");
```

```
63 });
```

TypeScript api/tests/routes.test.ts, lines 96–97

```
96 test("health: live is always ok; ready follows the database", async () => {
97   assert.deepEqual(await call("GET", "/health/live"), { status: 200, body: { status: "ok" } });
```

These are real HTTP requests with `fetch` against the real Express app, so they test routing, JSON parsing, status codes and the error handler, everything except SQL. SQL is the contract's job.

4.4 Running them

Command	Runs
`npm test` in api/	Everything; the PostgreSQL contract is skipped
`docker compose up -d db`, then `docker compose exec db createdb -U tasks tasks_test`	Creates a test database next to the real one (once)
`PGHOST=localhost PGUSER=tasks PGPASSWORD=tasks-dev-password PGDATABASE=tasks_test npm test`	Everything, including PostgreSQL (bash syntax; the README has PowerShell. Publish port 5432 first)
`npm run check`	Type-check, then test: what CI runs

4.5 Interview questions

What's a contract test?

One set of tests that every implementation of an interface must pass. It keeps fakes honest, so tests that use the fake still say something about the real thing.

Why listen on port 0?

The OS assigns a free port, so tests can run in parallel and alongside a dev server.

How do you keep database tests from damaging real data?

Run them only against a database whose name marks it as disposable, create it fresh in CI, and reset it before every test.

Part 5

Keeping it working

Dependencies pinned by lock files, version numbers that mean something, and CI on every push: the same three ideas as the robot arm's Appendix C.

5.1 package.json

JSON api/package.json, lines 1–28

```
{
  "name": "tasks-api",
  "private": true,
  "version": "1.1.0",
  "type": "module",
  "engines": {
    "node": ">=22"
  },
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "typecheck": "tsc --noEmit -p tsconfig.test.json",
    "test": "tsx --test \"tests/**/*.test.ts\"",
    "check": "npm run typecheck && npm test"
  },
  "dependencies": {
    "express": "^5.1.0",
    "pg": "^8.16.0"
  },
  "devDependencies": {
    "@types/express": "^5.0.0",
    "@types/node": "^22.15.0",
    "@types/pg": "^8.15.0",
    "tsx": "^4.19.0",
    "typescript": "^7.0.2"
  }
}
```

Line 4 **Semantic Versioning**: MAJOR.MINOR.PATCH. 1.1.0 added tests and refactored storage without changing the API's behaviour, so a minor version. CHANGELOG.md says what counts as breaking: the REST API and how the app is run.

Line 7 `engines` says which Node versions are supported; `.nvmrc` (22) says which one to use. CI tests 22 and 24.

Line 18 A **caret range**: any 5.x from 5.1.0 up, never 6. Under SemVer, that's everything that promises not to break.

Line 23 Types for the *oldest* supported Node. With types for 24, code could call a function Node 22 doesn't have and still type-check.

5.2 Lock files and npm ci

A range says what's allowed; `package-lock.json` records what was actually installed: the exact version and an integrity hash for every package, including the hundreds pulled in indirectly. It's committed.

	`npm install`	`npm ci`
Reads	package.json; updates the lock	the lock only
If they disagree	rewrites the lock	fails
node_modules	updated in place	deleted and rebuilt

	`npm install`	`npm ci`
Use it	adding or upgrading a package	CI, Dockerfiles, a fresh clone

Both Dockerfiles and every CI job use `npm ci`, so what's tested is exactly what ships.

5.3 The workflow

YAML ../../.github/workflows/fullstack-node-postgres.yml, lines 3–15

```yaml
name: full stack node-postgres

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Line 6 Every push to every branch, and every pull request: no path filters, so required checks always report.

Line 15 A new push to the same branch cancels the run still going for the old one.

YAML ../../.github/workflows/fullstack-node-postgres.yml, lines 18–56

```yaml
  api:
    name: api (Node ${{ matrix.node }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        node: [22, 24]
    defaults:
      run:
        working-directory: Full Stack/node-postgres/api
    # A throwaway PostgreSQL for the contract tests. The database name ends in _test, which is
    # what tests/postgres.test.ts requires before it will empty a table.
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_USER: tasks
          POSTGRES_PASSWORD: tasks-ci-password
          POSTGRES_DB: tasks_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U tasks -d tasks_test"
          --health-interval 5s --health-timeout 3s --health-retries 10
    env:
      PGHOST: localhost
      PGUSER: tasks
      PGPASSWORD: tasks-ci-password
      PGDATABASE: tasks_test
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: ${{ matrix.node }}
```

```
          cache: npm
          cache-dependency-path: Full Stack/node-postgres/api/package-lock.json
      - run: npm ci
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
```

Line 24 A **matrix**: the job runs twice, once per Node version. `fail-fast: false` lets both finish, so you see whether a failure is version-specific.

Line 30 A **service container**: a real, empty PostgreSQL started next to the job and waited for with its health check.

Line 45 Ends in `_test`, so the PostgreSQL contract runs here instead of skipping.

Line 51 Caches npm's download cache, keyed on the lock file.

YAML ../../.github/workflows/fullstack-node-postgres.yml, lines 74–99

```
  images:
    name: docker images
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/node-postgres
    steps:
      - uses: actions/checkout@v7
      # The same build docker compose runs locally, so a broken Dockerfile shows up here.
      - run: docker compose build

  audit:
    name: audit runtime dependencies
    runs-on: ubuntu-latest
    strategy:
      matrix:
        project: [api, frontend]
    defaults:
      run:
        working-directory: Full Stack/node-postgres/${{ matrix.project }}
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: 22
      - run: npm audit --omit=dev --audit-level=high
```

`docker compose build` catches a broken Dockerfile. `npm audit --omit=dev` fails on a known high or critical vulnerability in anything that ships; dev tools never reach the image, so they aren't counted.

5.4 Dependabot

YAML ../../.github/dependabot.yml, lines 28–55

```yaml
  # ---- Full Stack ----
  - package-ecosystem: npm
    directories:
      - "/Full Stack/node-postgres/api"
      - "/Full Stack/node-postgres/frontend"
      - "/Full Stack/dotnet-sqlserver/frontend"
      - "/Full Stack/go-php-mongo/web"
    schedule:
      interval: weekly
      day: monday
    open-pull-requests-limit: 10
    commit-message:
      prefix: "full stack"
    groups:
      # React and its types are released together.
      react:
        patterns: ["react", "react-dom", "@types/react", "@types/react-dom"]
      # Vite and its React plugin must be upgraded together: each new major needs the other.
      vite:
        patterns: ["vite", "@vitejs/*"]
      # The same for Stripe's two packages.
      stripe:
        patterns: ["@stripe/*"]
      minor-and-patch:
        update-types: [minor, patch]
    ignore:
      - dependency-name: "@types/node"
        update-types: ["version-update:semver-major"]
```

Every Monday, Dependabot opens pull requests for outdated packages, minor and patch updates bundled into one. Some packages must move together: React and its types are released together, Vite's React plugin needs a matching Vite, and Stripe's two packages need each other. A **group** puts each set in one pull request. A separate entry keeps the Docker base images (`node:22-alpine`, `nginx:1.31-alpine`, `postgres`) current, monthly. Each pull request runs the workflow above, so an update that breaks something shows up red before it's merged.

Green isn't always right

The first Dependabot run offered `node:25-alpine` for every Dockerfile, and the images built, so CI was green, and they were merged. But Node 25 is an odd-numbered release: short-lived, and already past its end of life. The project runs on the long-term-support line (22, matching `engines` and `.nvmrc`), so the images went back to 22, and Dependabot now skips Node majors for the images. Moving to the next LTS (24) is a deliberate change of `engines`, `.nvmrc` and CI together, not a base-image bump.

Two rules come out of it. A passing check only proves what it checks: building an image isn't running it. And for every update, read what changed and ask whether you want it, not only whether it's green.

5.5 Interview questions

^5.1.0 vs ~5.1.0?

Caret allows minor and patch updates (5.x); tilde allows patch only (5.1.x). For 0.x versions, the caret is as strict as the tilde, because below 1.0.0 a minor bump may break.

Why commit package-lock.json?

So every install, on any machine, gets the same tree. Without it, two builds from the same commit can differ.

Dependabot offers a major version and CI is green. Merge it?

Not automatically. Green means the checks passed, not that the change is wanted: Node 25 built fine but is past end of life. Read the release notes, check it fits the support policy (LTS here), and if it's a version you'll skip, tell Dependabot to ignore it.

Why test against PostgreSQL in CI if the routes are tested with a fake?

The fake can't check SQL. A typo in a query, a wrong alias or a type mismatch only shows up against the real database.